

DAG tokenizer using greedy sem cov

yuanyuan.zheng

August 2026

1 Introduction

1.1 Background

- *EHR Foundation Model (FM) arises nowadays, with potential to be assistive clinical decision tool*
- *they often use standardized clinical vocabularies such as ICD, RXNORM, SNOMED, LONIC*
- *standardized clinical vocabularies /ontologies DAG, are used to normalize clinical naming, and improve the interoperability*
- *SNOMED CT is a DAG, comprehensive 400k + concepts (covering very broad clinical domains, disorders, medication, morphologies etc...) and semantically rich (composed by 100+ relationship types, have their own way to compose machine readable semantic expression), mappable to other clinical terminologies.*

Electronic health record (EHR) foundation models (FMs) have recently emerged as promising assistive tools for clinical decision support (e.g. Li et al., 2020, BEHRT; Yang et al., 2023, TransformEHR). Because clinical events are recorded heterogeneously across institutions, such models are typically trained over standardized clinical vocabularies – ICD, RxNorm, SNOMED CT, LOINC – rather than free text, so that the same clinical fact is represented the same way regardless of where it was recorded **[CITE: standardized clinical vocabulary / interoperability reference]**. Several of these vocabularies are not flat code lists but ontologies structured as directed acyclic graphs (DAGs), which is what lets them normalize clinical naming while also supporting interoperability across terminologies. SNOMED CT is the clearest example: it comprises more than 400,000 concepts covering disorders, procedures, medications, morphologic findings and more, connected by over 100 relationship types that let each concept be expressed as a machine-readable logical composition of others, and it is mappable to most other major clinical terminologies **[CITE: SNOMED International concept model / release statistics]**. **[DOUBT: worth cross-checking against Su et al. (2025, MedTok)’s own count – they report 303,325 SNOMED CT concepts specifically (out of 617,490 codes total across 8 systems: ICD-10-CM 81,184, RxNorm 81,151, ICD-10-PCS**

61,644, NDC 54,560, ICD-9 18,365, CPT 10,602, ATC 6,659) – likely a different SNOMED edition/extension than your 400k figure; worth a one-line reconciliation rather than leaving two different SNOMED counts unexplained in the same paper.]

1.2 Gap

- *training FM needs either large amount of patient data (but patient data is difficult to get)*
- *or need adequate size of clinical vocabularies. (but medical vocabularies are large size)*
- *Current methods are:*
- *(cite) LLM byte-pair on categorical medical concept: data-driven, no semantic preservation, highly dependant to data quality and concept orders.*
- *(cite) keep highest prevalent top k concepts, highest entropy contribution top k concepts: data-driven, loss of considerable clinical information.*
- *(cite) truncate/reduce leaf concepts to their parents concepts: data-driven, loss of medical granularity.*
- *(cite) or just keep all concepts as it is, if we have lots of patient data. but since concept usage distribution are usually long tail, many rare concepts can not be learnt in a proper way.*
- *(cite) learnable embedding space codebook: loss of controllability and interpretability*
- *all of above mentioned methods are ignoring the semantic richness of clinical vocabulary DAG when try to deal with vocabulary size.*

Training an EHR FM directly on top of such a vocabulary creates a tension between two resources that are each expensive on their own. Learning a reliable representation for every concept in the vocabulary requires a correspondingly large amount of patient data, which is costly to obtain and often access-restricted [DOUBT: do you want to cite/quantify this, e.g. typical cohort sizes needed vs. available, or leave it as a general statement?]. Restricting training to a vocabulary small enough to be learnable from the data actually at hand is not free either, precisely because clinical vocabularies are large by construction and concept usage in real EHR data follows a long tail: most concepts occur too rarely for a model to learn a stable representation for them. Su et al. (2025, MedTok) make the same observation independently – “low-frequency codes suffer from poor embeddings, reducing performance on underrepresented conditions” – which is a second, recent voice for this exact claim, though neither their paper nor this one currently backs it with an actual frequency distribution [DOUBT: would strengthen this sentence a lot to show your own long-tail plot from the mapped-concept usage data, if you have it, rather than relying on two qualitative claims (yours and MedTok’s) with no numbers behind either].

Existing strategies for shrinking the effective vocabulary size largely leave this semantic structure on the table. Data-driven subword approaches, such as byte-pair encoding applied to categorical medical codes, merge concepts purely by co-occurrence statistics: they preserve no explicit semantics and the resulting vocabulary is highly sensitive to data quality and to the order in which concepts are seen (Sennrich et al., 2016; Kudo & Richardson, 2018, SentencePiece). Frequency- or entropy-based top- k selection keeps only the most prevalent, or highest-information-contributing, concepts, discarding a considerable amount of clinically relevant but rarer information **[CITE: prevalence/entropy-based vocabulary pruning in clinical ML]**. Truncating or rolling leaf concepts up to their ontology parents reduces vocabulary size at the direct cost of clinical granularity – a specific diagnosis collapses into a broader category **[CITE: ontology roll-up / ICD truncation approach]**. Keeping every concept exactly as-is sidesteps these losses only when enough patient data is available, and otherwise simply defers the long-tail problem to the model **[DOUBT: is there a citation you have in mind for this "no reduction" baseline, or is it meant as the trivial comparison point rather than a cited method?]**. Finally, learnable embedding-space codebooks (e.g. vector-quantization-style tokenizers, Van den Oord et al., 2017) can compress the vocabulary without hand-designed rules, but at the cost of controllability and interpretability: the resulting tokens no longer correspond to identifiable, auditable clinical concepts. **[DOUBT: Is the sharpest way to state the gap "none of these use the DAG structure itself," or the stronger claim "none of these produce a token set that is both semantically motivated and inspectable/controllable"? The second framing lines up more directly with the contribution list below – your call on which to lead with.]**

Closest related work. Su et al. (2025, MedTok) is the one method in this space that no longer fits the "ignores graph structure entirely" version of the critique above: it feeds each code’s biomedical knowledge-graph neighborhood, alongside its text description, into a multimodal encoder, then vector-quantizes the result into a shared codebook (learned, size on the order of 10^4 entries) to produce each code’s token sequence. So the sharper distinction is not "uses the graph vs. doesn’t," but *how*: MedTok’s tokens are indices into a learned, continuous codebook, opaque by construction – there is no way to read off which relationships in the graph produced a given token. Our tokens are, by contrast, selected directly and deterministically from the graph itself (Section 3), and every token a concept resolves to comes with an explicit, inspectable trace of which relationships were followed to reach it. **[DOUBT: this paragraph is my synthesis of the distinction, not something either paper states outright – worth double-checking it reads as fair to MedTok (it’s a real, well-evaluated method, not a strawman) rather than as dismissive.]**

1.3 Our method and contribution

- *applicable to DAG knowledge graph.*
- *tokens selection: Represent concepts by smaller vocabulary size using*

greedy token selection approach

- *way to tokenize: Represent concepts by a tree/subgraph preserving the structure of the concepts the semantic value.*
- *By apply to SNOMED CT concepts used in our local institut real data, we show our method can control how we tokenize and with what we tokenize (controllability and interpretability)*
- *(maybe) if any mapping inter-ontology exist, we can use concepts in 1 ontology to represent/tokenize concepts in the other ontologies (interoperability)*

We propose a token-selection and tokenization method that operates directly on a concept DAG such as SNOMED CT. First, we select a small set of tokens using a greedy, submodular semantic-coverage objective (Section 3), which fixes the target vocabulary size in advance rather than as a side effect of a training run. Second, rather than representing a concept as an unordered bag of the tokens it maps to, we represent it as a *context tree*: a small subgraph that preserves which relationship each token was reached through, so that, for instance, a token for "kidney" and a token for "left" stay attached to the same anatomical-site branch instead of being silently merged with an unrelated "right adrenal gland" branch found elsewhere in the same concept **[DOUBT: this kidney/adrenal example is carried over from your existing PDF draft – keep it, or swap in a real example from the HUG-mapped concepts once we have one handy?]**. Third, applying this method to the SNOMED CT concepts mapped by our institution’s semantician team, we show that, unlike the alternatives above – including MedTok, the closest recent method that also uses graph structure – the resulting tokenization is directly controllable and interpretable: for any given concept, a domain expert can inspect exactly which relationships were followed to reach which token.

[NOTE: Per your feedback: the inter-ontology point (last bullet above) is not being claimed as a contribution of this paper – it’s mentioned only in Future Work (Section 7) now, not developed here.]

2 Background

2.1 Directed acyclic graphs

- *DAG (directed acyclic graph)*

A directed acyclic graph (DAG) is a directed graph $G = (V, E)$ that contains no directed cycle: there is no sequence of edges that both starts and ends at the same vertex while always following edge direction. Equivalently, G admits at least one topological ordering of V in which every edge points from an earlier to a later vertex in the ordering. Acyclicity is what makes "child" and "parent" well defined for any two vertices connected by a directed path: a vertex reached by following edges forward is strictly more specific than, and can never be an ancestor of, the vertex it was reached from.

2.2 SNOMED CT

- *Number of concepts, number of distinct relations,*
- *DAG nature*
- *semantic value from relation types and concept connectivities (concepts are defined by the others in the DAG)*

SNOMED CT is one such graph. It comprises $|V| \approx 400,000$ concepts [CITE: SNOMED International release statistics] spanning disorders, procedures, findings, body structures and substances, connected by [RESULTS NEEDED: exact number of distinct relationship types in the current release / extension in use] relationship types.

Two properties of this graph matter for the rest of the paper. First, it is acyclic. Second, its edges carry semantic value beyond hierarchy. One relationship type, `IS_A`, encodes generalization/specialization – the parent/child relation defined above. Every other relationship type (for example *finding site*, *associated morphology*, *has active ingredient*) attaches an attribute to a concept by pointing to another concept, rather than by storing a value locally. As a consequence, a concept’s meaning is not contained in the concept itself but constructed from its position and connections inside the DAG – two concepts that look unrelated by name can share most of their meaning if they are reached through the same relationships to the same targets, and conversely two occurrences of the same relationship type from the same concept can point to entirely different, non-interchangeable aspects of it. This is exactly the structure our method is built to exploit: Section 3 defines a coverage score and a tree representation that treat `IS_A` edges and non-`IS_A` edges differently for precisely this reason.

In practice, candidates are not drawn from all of V but from a bounded neighborhood of the mapped concepts M (Section 3.1 gives the exact construction). Under the current depth budget (`max_dist_candidate` = 9), this candidate/working subgraph contains **75,964** concepts, spread across **2 weakly connected components**. *[NOTE: worth a short remark in the paper: this is barely larger than the candidate-pool size computed under the old $D = 3$ setting ($\sim 75,634$), despite D growing from 3 to 9 – i.e. the D -hop ego-graph around M saturates fast, most of the reachable neighborhood was already captured at $D = 3$. Also worth flagging: 2 weakly connected components means the candidate graph is not a single connected structure – does that split correspond to something meaningful (e.g. two disjoint clinical domains in M), or is it a couple of stray/isolated concepts? Good to know before Section 4 describes the pool.]*

2.3 Working vocabulary and notation

Let $M \subseteq V$ denote the subset of SNOMED CT concepts mapped by our institution’s semantic team to locally used clinical concepts ($|M| \approx 40,000$), and let $T \subseteq V$, $|T| = k$, denote a *token set*: a small subset of concepts chosen to represent every concept in M (Section 3).

3 Method

3.1 Candidate pool and depth budget

A single depth budget D plays three roles in this method, all controlled by the one hyperparameter `TokenizerParam.max_dist_candidate` in the implementation: it bounds how far a candidate token may be from the mapped concepts it can help represent, it bounds how deep a context tree (Section 3.3) is allowed to grow, and it bounds the horizon over which the coverage score below is computed. Concretely, the candidate pool is

$$V_D = M \cup \{u \in V : u \text{ is reachable from some } c \in M \text{ by at most } D \text{ relationships}\},$$

obtained in practice as the union of the D -hop ego-graph (following outgoing edges) around every mapped concept – see Section 2 for its current size. Token selection (Section 3.2) only ever considers $t \in V_D$: a candidate that no mapped concept can reach within D hops has zero possible contribution to the objective below and is never evaluated. **[DOUBT: Using one shared D for all three roles is a simplifying design choice worth calling out explicitly – a reviewer may ask why the candidate radius, the tree depth, and the selection horizon are tied together rather than treated as three independent hyperparameters. Is that coupling intentional/load-bearing, or just a current implementation convenience?]**

3.2 Greedy token selection

- *Greedy token selection based on semantic coverage with lambda as distance penalty (add illustration)*

For $u \in V$, $T \subseteq V$ and horizon $h \in \{0, \dots, D\}$, let $S_h(u, T) \in [0, 1]$ denote how well u is covered by T when at most h further relationships may be traversed. The base case is

$$S_0(u, T) = \mathbf{1}[u \in T],$$

and, extending the recursion with a per-hop distance penalty $\lambda \in (0, 1]$ *[NOTE: confirmed by you: this λ term is your own addition on top of the original PDF draft, which only has the undiscounted ($\lambda=1$) version of Eq. (1) – well spotted catching that gap.]*,

$$S_h(u, T) = \begin{cases} 1, & u \in T, \\ 0, & u \notin T \text{ and } d^+(u) = 0, \\ \frac{\lambda}{d^+(u)} \sum_{(u,r,v) \in \text{Out}(u)} S_{h-1}(v, T), & \text{otherwise.} \end{cases} \quad (1)$$

A token is absorbing (its coverage is 1 at every remaining horizon); every other node’s coverage is the λ -discounted average coverage of its out-neighbors. Setting $\lambda = 1$ recovers the undiscounted score, in which a token found j hops away

and a token found at the root contribute equally; for $\lambda < 1$, a token found j hops away instead contributes λ^j , so the score also rewards *proximity* to a token, not only reachability. The overall objective is the mean coverage of the mapped concepts at the full horizon D :

$$F_D(T) = \frac{1}{|M|} \sum_{c \in M} S_D(c, T), \quad F_D(\emptyset) = 0, \quad 0 \leq F_D(T) \leq 1.$$

F_D is normalized, monotone, and submodular in T (the marginal gain of adding a token never increases as more tokens are already selected), which follows from writing F_D as a positive average, over random depth- D walks from each $c \in M$, of the indicator that the walk hits T . Submodularity gives the greedy algorithm the classical approximation guarantee

$$F_D(T_k^{\text{greedy}}) \geq \left(1 - \frac{1}{e}\right) \max_{|T| \leq k} F_D(T),$$

and, more importantly for a candidate pool of tens of thousands of nodes, lets us use *lazy* greedy instead of exact greedy without changing the selected set: each candidate’s previously computed marginal gain is a valid upper bound on its current gain, so at every step it suffices to re-verify only the best stale candidate against the next-best stale bound, rather than recomputing every remaining candidate. Adding one candidate t only changes S_h at t itself and at vertices that can reach t within D hops in reverse, so a single candidate’s exact marginal gain can be computed by propagating a sparse delta backward through the incoming-edge structure instead of recomputing the full $O(D(|V| + |E|))$ recursion from scratch.

[FIGURE PLACEHOLDER – illustration of the λ -decayed coverage propagation, e.g. the small worked example from your PDF ($u \rightarrow \{t_1, v\}, v \rightarrow \{t_2, z\}$): show S_h propagating backward from t_1, t_2 toward u , and how the contribution shrinks by a factor of λ per hop.]

Figure 1: Semantic coverage propagation (to be replaced with an actual figure/diagram).

3.3 Tokenizer

- *tokenizer (say why flatten is not good, add illustration, resulting context trees)*

Once a token set T is fixed, a mapped concept $c \in M$ is not represented by the flat, unordered set of tokens reachable from it. A flat representation throws away exactly the information that non-ISA relationships were designed to carry: if a concept has two independent *finding site* occurrences, each qualified by

its own *laterality*, a flat set of tokens {kidney, adrenal gland, left, right} can no longer tell which laterality belongs to which site. **[DOUBT: keeping this exact kidney/adrenal example from your PDF – confirm, or swap for a real HUG-mapped concept once we pick an illustrative one for the figure below.]**

We instead recursively expand each mapped concept c into a *context tree*. IS_A relationships are transparent and continue the search in the current context; every other relationship type opens a fresh subcontext, so that two independent occurrences of the same non-IS_A relationship type are kept in two separate subcontexts. Formally, $\text{EXPAND}(u, q, d)$, starting from $\text{EXPAND}(c, q_0, 0)$:

1. if $u \in T$, attach u as a token of the current context q and stop this branch;
2. otherwise, if $d = D$ or u has no outgoing edges, mark this branch *uncovered* in q and stop;
3. otherwise, for every outgoing edge (u, r, v) : if $r = \text{IS_A}$, continue in the same context q ; otherwise, open a new subcontext of q labelled by r and continue there.

A token located exactly at depth D is still accepted (case 1 is checked before case 2). The resulting tree keeps only T as its vocabulary; its subcontext structure records exactly which relationship chain each token was reached through, which is what preserves the kidney/left vs. adrenal gland/right distinction above.

[FIGURE PLACEHOLDER – side-by-side: flat token set vs. context tree for a real mapped concept with ≥ 2 non-IS_A relationship occurrences, e.g. the kidney/adrenal-gland example, or a concept pulled directly from the HUG-mapped data.]

Figure 2: Context-tree tokenization example (to be replaced with an actual figure).

3.4 Evaluation metrics

- *metrics used and how they are computed,*
- *3.1: Efficiency: conciseness, tree_complexity*
- *3.2: distance_score: Fidelity*
- *3.3: discriminative power: uniqueness_entropy, unique_rate*
- *3.4: semantic content: semantic_coverage, uncovered_rate, unk_branch_rate*
- *and also say either they are hogher or lower better (interpretation)*

Every metric below is computed once the context trees for all of M have been built against a fixed T ; they evaluate the *realized* representation, as opposed to $F_D(T)$, which is the objective used to *select* T in the first place (Section 3.2).

Efficiency. *Conciseness* is the mean number of token leaves per concept’s tree; *tree complexity* is the mean number of contexts (root plus subcontexts) per tree. Lower is better for both: a concept represented by fewer tokens and fewer subcontexts is cheaper to encode and easier to inspect.

Fidelity. *Distance score* converts the mean hop-distance at which tokens are found (uncovered branches counted at $D + 1$ rather than dropped) into a $(0, 1]$ similarity, $1 - \bar{d}/(D + 1)$. Higher is better: 1.0 means every branch found a token immediately.

Discriminative power. *Uniqueness entropy* is the normalized Shannon entropy of context-tree signatures across M : it is low when many concepts collapse onto the same tree shape (i.e. become indistinguishable under T) and high when signatures are spread out. *Unique rate* is its per-concept decomposition – the fraction of concepts whose own signature is not shared by any other concept. *Exact rate* – the fraction of mapped concepts that are themselves selected as a token, i.e. a 0-hop, exact representation – also belongs here per your feedback: a concept selected as its own token is maximally discriminable from every other concept (its signature cannot collide with anything). Higher is better for all three.

Semantic content. *Semantic coverage* is $F_D(T)$ itself, always reported at $\lambda = 1$ regardless of the λ a candidate list was *selected* under **[DOUBT: confirm this is still the evaluation convention you want – i.e. selection sweeps λ , but the reported metric always scores coverage at $\lambda = 1$, so every method is compared on the same undiscounted notion of "did it reach a token at all"]**. *Uncovered rate* is the fraction of concepts with zero tokens anywhere in their tree – not represented at all. *Unk rate* is the (weaker) fraction of concepts with *at least one* uncovered branch somewhere in their tree, i.e. partially unrepresented; by construction uncovered rate $\leq$ unk rate. Higher is better for semantic coverage; lower is better for both uncovered rate and unk rate.

3.5 Baselines

- *Chosen baseline for token selection (all DAG compatible)*
- *4.0: k random*
- *4.1: degree based: Most children, Highest degree, Highest degree with $D \downarrow = 1$*
- *4.2: centrality based: Closeness centrality, Page rank, Personalized page rank*
- *4.3 ours, and also our ablated version: Discrete set cover*

All baselines below rank the full candidate pool V_D once and are then truncated with the top- k prefix of that ranking, exactly as the greedy method’s own selection history is truncated – so every method in Section 4 is compared at matched k .

Random. *k-random*: for each k , an independent uniform sample of k candidates without replacement, repeated over multiple seeds and averaged, since it is not a single ranking that can be truncated the same way as the others.

Degree-based. *Highest degree*: rank by number of distinct predecessors within D hops, over every relationship type. *Highest degree, $\Delta \leq 1$* : the same score restricted to direct (1-hop) predecessors. *Most children*: the same D -hop predecessor count, but restricted to the IS_A-only hierarchy edge subgraph – i.e. how many descendant concepts a candidate has in the hierarchy alone, ignoring every other relationship type.

Centrality-based. *PageRank*: the stationary distribution of a random walk with restart probability $1 - \alpha$ ($\alpha = 0.85$) [CITE: Brin & Page, PageRank]. *Personalized PageRank*: the same recursion but restarting uniformly on M instead of on all of V , making it the one centrality baseline that is M -aware rather than purely structural [CITE: personalized / topic-sensitive PageRank, e.g. Haveliwala]. *Closeness centrality*: the (in-degree-oriented, Wasserman–Faust improved) reciprocal of the mean shortest-path distance to a candidate from every node that can reach it [CITE: Freeman, closeness centrality]. [NOTE: Per your feedback, eigenvector centrality has been removed from this baseline list entirely – it doesn’t work on a DAG (its Perron–Frobenius uniqueness guarantee requires strong connectivity), so it is no longer part of this draft even as a flagged/caveated option.]

Ours, and its hard-greedy ablation. Our method is the lazy-greedy maximizer of F_D from Section 3.2. As an ablation, we also report a *discrete set-cover* selector that keeps the same lazy/CELF greedy mechanism – and the same $(1 - 1/e)$ approximation guarantee – but replaces the soft, λ -decayed, multi-hop-averaged coverage $S_h(u, T)$ with a binary ”is c within D hops of some token in T ” indicator, i.e. classical maximum-coverage greedy applied to the same candidate pool. Comparing the two isolates what the soft, distance-aware formulation of Eq. (1) buys over textbook hard-coverage greedy, rather than comparing against a strawman.

3.6 Expert acceptability evaluation

- *evaluation of acceptability of tokenized version versus ooriginal concept, by clinical expert.*

In addition to the automatic metrics above, we evaluate tokenization quality directly with a clinical expert: for a sample of mapped concepts, the expert is shown the original concept alongside its context-tree tokenization (Section 3.3) and rates whether the tokenized version still conveys an acceptable representation of the original concept’s meaning. [NOTE: per your feedback, this is now firm in-paper content (no longer ”(maybe)”), and it is intrinsic quality evaluation, distinct from the downstream/assistive-clinical-decision-task validation named in Section 6 – the two are not the same thing and this draft now keeps them separate.] [DOUBT: still need from you: how many concepts/experts, what rating scale (binary acceptable/not, Likert, free-

text justification?), and whether inter-rater agreement is reported if there is more than one expert – these design choices affect how this subsection should be written up and what Section 4 needs to report alongside the automatic metrics.]

4 Results

4.1 Data

- *use all concepts SNOMED used and mapped by our semantician in HUGData*

We use every SNOMED CT concept mapped by our institution’s semantician team in HUGData as the mapped set M . *[NOTE: "HUGData" is used here exactly as it appears in your own `config.py` path names – flag if this should instead be spelled out or anonymized for the paper.]*

4.2 Candidate pool

- *how to get subgraph and narrow down the candidate pool, then give the number 400k - 70k*

Candidate tokens are restricted to the D -hop neighborhood of M (Section 3.1), which narrows the pool from $|V| \approx 400,000$ SNOMED concepts to **75,964** concepts under the current setting ($D = 9$), spread across **2 weakly connected components**. *[NOTE: also noted in Section 2: this is barely larger than the old $D=3$ figure ($\sim 75,634$) despite the much larger depth budget – the ego-graph around M saturates quickly.]*

4.3 General trend across k

- *General trend: k VS all metrics evolution*

Table 1 shows semantic coverage $F_D(T)$ (evaluated at $\lambda = 1$, Section 3.4) against k , for one representative member of each family (Section 3.5): *highest degree* (degree-based), *personalized PageRank* (centrality-based), our own selector at $\lambda = 1$ and, separately, the full λ sweep, the *discrete set-cover* ablation, and *k-random*.

Three things hold across the entire tested range ($k \in [500, 19,500]$, i.e. up to roughly half of $|M| \approx 40,000$), not just at the k -values tabulated above. First, every structured method – ours, both degree-based, and all three centrality-based baselines – reaches semantic coverage ≥ 0.999 within the tested range, while *k-random* does not: it only reaches 0.804 at the largest k tested, still well short of every structured method. Second, our own method’s envelope (best over $\lambda \in \{0.2, \dots, 1.0\}$) is the fastest of all five families to reach ≥ 0.999 coverage, doing so already at $k = 500$, the smallest budget tested – ahead of centrality ($k = 1,000$), degree ($k = 3,000$), the discrete-set-cover ablation (never reaches

k	500	1,000	5,000	10,000	19,500
Highest degree	0.981	0.985	1.000	1.000	1.000
Personalized PageRank	0.998	0.999	1.000	1.000	1.000
Ours, $\lambda = 1.0$	1.000	1.000	1.000	1.000	1.000
Ours, $\lambda = 0.8$	0.998	1.000	1.000	1.000	1.000
Discrete set-cover (ablation)	0.977	0.978	0.991	0.995	0.999
k -random	0.050	0.073	0.351	0.587	0.804

Table 1: Semantic coverage $F_D(T)$ (evaluated at $\lambda = 1$) vs. token budget k , one representative per family. Full λ sweep in Table 2.

0.999 within the tested range, topping out at 0.999 only at $k = 19,500$), and random (never reaches it). Third, k -random’s gap to every structured method does not close within the tested range: it is still roughly 0.2 below the nearest structured method at $k = 19,500$.

4.4 Comparison across baseline families

- *analysis of metrics among different families of candidate selection, and the characteristics:*
- *degree based: may select semantic overlapped concept, less semantic preservation, redundant selection in same branch(?)*
- *centrality based: random walk based, less semantic preservation (?), overlapped semantics(?), maybe less efficiency (?)*
- *ours*
- *in this part, for each family, i will plot the upper envelop curve of metric performance*

Degree-based. Within this family, *highest degree* (all relation types) dominates *most children* (IS_A only) on every metric we checked (distance score, conciseness, uniqueness entropy, unique rate) at every k we sampled, with one narrow exception: between $k \approx 2,500$ and $k \approx 4,500$, *most children* briefly has very slightly *higher* semantic coverage (e.g. 0.999 vs. 0.996 at $k = 3,000$) before *highest degree* overtakes it again by $k = 5,000$ and stays ahead for the rest of the range. *[NOTE: this local crossover is real, not a plotting artifact – verified directly against the parquet rows – but it’s narrow enough that I would state the headline claim as “highest degree dominates most children almost everywhere” rather than “always,” and mention the crossover as a footnote rather than lead with it. The pattern you predicted qualitatively (non-hierarchy relations carrying extra coverage signal a pure-IS_A heuristic misses) does hold as the overall story – just not exceptionlessly.]*

Centrality-based. *Personalized PageRank*, the one M -aware member of this family, reaches near-ceiling coverage fastest within the family ($k = 1,000$), ahead of plain PageRank and closeness centrality – consistent with the idea that

restarting on M rather than uniformly should help when the objective is itself about covering M .

Ours vs. every baseline family, on coverage speed. At small k (500–1,000, i.e. roughly 1–2.5% of $|M|$), our λ -sweep envelope is the only family already at or above 0.998 semantic coverage; every baseline family needs a larger k to catch up (Table 1).

Discriminative power (uniqueness entropy / unique rate). [DOUBT: your hypothesis that degree-based methods would show *lower* discriminative power than ours, from selecting semantically-overlapping candidates, is only partly borne out: at $k = 1,000$, the best degree-based member reaches unique rate 0.588, close to our envelope’s 0.594 – not the large gap the hypothesis predicted. Where a real gap does show up is at $k = 15,000$: centrality-based baselines slightly *overtake* our envelope on both uniqueness entropy (0.989 vs. 0.987) and unique rate (0.903 vs. 0.893) – the one place in this sweep where a baseline family is not dominated by ours. Worth stating honestly rather than glossing over; it doesn’t undermine the coverage-speed story above, but it means “ours wins on every metric at every k ” would not be an accurate claim.]

Ablation: soft vs. hard coverage. The discrete set-cover baseline (Section 3.5) – same lazy/CELF greedy mechanism and approximation guarantee as our method, but a binary rather than λ -decayed coverage signal – is the slowest of all *structured* methods to approach ceiling coverage: it never reaches ≥ 0.999 within the tested range (topping out at 0.999 exactly at $k = 19,500$), behind every degree-based and centrality-based baseline as well as our own method. This is a direct empirical confirmation that the soft, distance-aware formulation of Eq. (1) buys something concrete over classical hard-coverage greedy, not just a theoretical nicety.

4.5 Effect of the distance penalty λ

- *analysis of metrics between our methods, with respect to impact of lamda*
- *extreme case: lam = 0.0: force to choose the mapped concepts.*
- *trade off lambda vs diff metrics.*
- *token reuse rate (need to be implemented)*
- *point out: best config depends on personal choice.*

Checking every one of the 39 tested k values (not just the $k = 10,000$ snapshot in Table 2), three distinct patterns emerge, not two:

Monotonically improving toward $\lambda = 1$ (100% of k values). Semantic coverage and unk rate improve monotonically as λ increases from 0.2 to 1.0, at *every single* tested k , always best at $\lambda = 1.0$. Uncovered rate follows the same direction but saturates to exactly 0 well before $\lambda = 1.0$ is reached: by $\lambda \geq 0.6$ it is already 0 at every $k \geq 1,500$, and even $\lambda = 0.2$ reaches 0 by $k = 5,000$.

λ	0.0	0.2	0.4	0.6	0.8	1.0
Semantic coverage	0.245	0.996	0.999	1.000	1.000	1.000
Unk rate	0.782	0.131	0.054	0.021	0.012	0.000
Distance score	0.260	0.804	0.838	0.855	0.863	0.728
Conciseness	2.591	9.985	8.940	7.971	7.411	11.500
Exact rate	0.217	0.199	0.183	0.169	0.155	0.185

Table 2: Effect of λ on five metrics at $k = 10,000$ (representative of the pattern at every k we checked – see prose for how consistently each pattern holds across the full $k = 500$ – $19,500$ range).

Peaked at $\lambda \approx 0.8$ (large majority of k values). Distance score is maximized at $\lambda = 0.8$ at *all 39* tested k values, dropping at $\lambda = 1.0$ every time (e.g. $0.863 \rightarrow 0.728$ at $k = 10,000$). Tree complexity is minimized at $\lambda = 0.8$ at 38 of 39 k values; conciseness at 34 of 39; uniqueness entropy at 30 of 39; unique rate at 27 of 39 (with $\lambda = 0.6$ the runner-up choice in most of the remaining cases). This is the confirmed version of your outline’s prediction – these metrics really do trade off against raw coverage, and $\lambda = 0.8$ is consistently, not just occasionally, the best single setting for this group.

Its own pattern: exact rate. Exact rate does not fit either group above. Among the positive λ values, it is highest at $\lambda = 0.2$ at *every* tested k and decreases overall as λ increases toward 1 – moving in the *opposite* direction from semantic coverage and unk rate as λ increases. It is not perfectly monotonic on the way down (e.g. at $k = 10,000$, $\lambda = 1.0$ ’s 0.185 is a partial rebound above $\lambda = 0.8$ ’s 0.155), but $\lambda = 0.2$ is the single best positive- λ choice for this metric without exception in the sweep.

The degenerate case $\lambda = 0$: proven, and confirmed empirically. As derived in Eq. (1): at $\lambda = 0$, every “otherwise” term vanishes, so $S_h(u, T) = S_0(u, T) = \mathbf{1}[u \in T]$ at *every* horizon, and $F_D(T)|_{\lambda=0}$ reduces exactly to $\text{exact_rate}(M, T)$ – selecting at $\lambda = 0$ optimizes a fundamentally different, narrower objective than coverage. This is not just an algebraic curiosity: it is exactly what the sweep shows. Across all 39 k values, the $\lambda = 0$ selection has (a) *strictly the highest* exact rate of every $\lambda \in \{0, 0.2, \dots, 1.0\}$ at every single k (e.g. 0.217 at $k = 10,000$, vs. 0.199 for the next best, $\lambda = 0.2$) – exactly as predicted, since it is directly what $\lambda = 0$ selection optimizes – and (b) *strictly the lowest* semantic coverage of every λ at every single k , dramatically so at small k : 0.012 at $k = 500$, actually *below* k -random’s 0.050 at the same budget, and still only 0.491 at the largest tested $k = 19,500$ (vs. ≥ 0.999 for every $\lambda \geq 0.2$). [NOTE: note for precision: the selection-time objective at $\lambda = 0$ literally equals *exact_rate*, but the reported *semantic_coverage* column is always evaluated at $\lambda = 1$ (Section 3.4’s convention) – so the two numbers in the table are not the same quantity computed twice; the point is that a token set optimized for *exact_rate* performs catastrophically under the standard $\lambda = 1$ coverage evaluation everyone else is judged by.]

Token reuse rate. [DOUBT: confirmed not yet implemented in `eval.py` – this subsection stays a placeholder until that metric exists in code; happy to help define/implement it once you specify what "reuse" should mean here (e.g. how many distinct mapped concepts a single token ends up representing, across the whole context-tree population).]

Practical takeaway. Given the above, $\lambda \approx 0.8$ is a reasonable default when concise, nearby representations matter as much as raw coverage: it costs essentially nothing in semantic coverage or unk rate relative to $\lambda = 1$ (both already at or near ceiling by moderate k), while giving the best distance score, tree complexity, and (usually) conciseness and discriminative power in the whole sweep. $\lambda = 1$ remains the right choice if raw coverage at the smallest possible k is the only thing that matters, or if a downstream use case specifically wants the highest possible exact-match rate among large- λ settings. [DOUBT: "depends on personal choice" – happy to sharpen this into an explicit recommendation/decision rule if you tell me what the downstream FM actually optimizes for (e.g. embedding-table size vs. per-concept fidelity).]

5 Discussion

- *Empty for the moment*

[DOUBT: Nothing drafted as prose here yet, per your note. Once Section 4 has real numbers, this section should close the loop back to Section 1's motivation. Suggested threads to pick up:]

- How much smaller a vocabulary does structured (greedy / degree / centrality) selection buy over the "keep everything" or "random" baselines from the Introduction's gap paragraph, at what cost in semantic coverage or distance score – i.e. turn Section 4.3/4.4 into a direct answer to "does this solve the FM vocabulary-size problem better than the alternatives listed in the Introduction."
- What the $\lambda = 0$ result (Section 4.5) implies practically: since λ trades off raw coverage against proximity/conciseness, is there a principled way to pick λ for a given downstream FM, or is it genuinely a design knob left to the practitioner (tying back to your Results outline's closing point)?
- Revisit the controllability/interpretability claim from Section 1's contribution list, and the expert acceptability results from Section 3.6: what does a reader/expert actually get to inspect or judge that a codebook-based tokenizer would not let them inspect? A short worked example (reusing the context-tree figure from Section 3.3) would make this concrete rather than asserted.

6 Limitations

6.1 Graph structure constraints

- *Graph structure constraints – incomplete semantic definition, which our tokenizer can not do shit*

The context tree and the coverage score (Section 3) can only ever reflect the relationships that are actually encoded in the graph. Where SNOMED CT’s own definition of a concept is incomplete – a relevant attribute simply not modeled as an outgoing edge – the tokenizer cannot compensate for this: an under-specified concept will look under-specified in its context tree regardless of which token set T is chosen. **[DOUBT: I read “our tokenizer can not do shit” as “our method has no way to fix or detect this,” which is what the sentence above says – flag if you meant something more specific, e.g. a particular class of missing relationships you’ve already run into.]**

6.2 Transductive method

- *transductive method for token selection*

Token selection (Section 3.2) is transductive: it optimizes coverage for a fixed, known M , and a token set selected for one institution’s mapped concepts is not guaranteed to transfer to a different mapped set without re-running selection.

6.3 No external validation on a clinical task

- *No external validation on clinical task*

This paper evaluates tokenization quality intrinsically (the metrics of Section 3.4) and, directly, via clinical-expert acceptability ratings of the tokenized representation against the original concept (Section 3.6). What it does not do is validate the tokenization on an actual downstream, assistive clinical decision task – i.e. show that a model trained on this reduced vocabulary performs a real clinical decision-support function (diagnosis support, risk prediction, etc.) at least as well as one trained on an alternative vocabulary. *[NOTE: per your feedback, this is a distinct claim from the expert acceptability evaluation in Method – that one is about representation fidelity judged by an expert, this one is about end-task clinical utility, and only the first is in scope for this paper.]*

6.4 Requires a DAG

- *Not working on Non-DAG*

The tokenizer’s core mechanism – IS_A transparency, non-IS_A subcontexts, and the coverage recursion’s reliance on well-defined finite hop distances – assumes an acyclic graph. It does not extend as-is to a vocabulary with cycles, or

to a general knowledge graph where the child/parent asymmetry of Section 2 does not hold.

7 Future Work

7.1 Inter-ontology tokenization

- *if mapping inter ontology snomed and others, can show case that we can tokenize concepts in 1 ontology with other concept tokens in other ontology*

Where a mapping between SNOMED CT and another ontology exists, the same selection and tokenization mechanism could be applied across ontologies: tokens selected in one vocabulary could represent concepts from another, extending the interoperability argument from Section 1 to the token level itself. *[NOTE: this is not claimed as a contribution of the current paper (see Section 1) – it lives here only, as you asked.]*

7.2 Redundancy and token swaps

[NOTE: pulled from your PDF’s §8 “Perspectives” – not in main.tex’s outline, but already sketched there.]

The greedy algorithm of Section 3.2 never removes a token once selected. A token can become nearly redundant once later tokens end up covering the same concepts it was originally selected for; a marginal-*loss*-based pass could identify and swap out such tokens after the fact.

7.3 Primitive vs. fully-defined concepts

[NOTE: also pulled from your PDF’s §8.]

Expanding a concept via its outgoing relationships (Section 3.3) implicitly assumes that doing so is lossless. For primitive SNOMED CT concepts – those not fully defined by their stated relationships – this may not hold. A reconstructibility factor $\alpha(u) \in [0, 1]$ ($\alpha(u) = 1$ for fully defined concepts, $\alpha(u) = \eta < 1$ for primitive ones) could discount the propagated coverage of Eq. (1) accordingly, while still treating a primitive concept selected *as its own token* as fully covered (coverage 1), since it is then preserved atomically rather than reconstructed from its relationships.

7.4 Combining with an ontology-similarity measure

[NOTE: also pulled from your PDF’s §8, which already cites a reference for this ([6] in your PDF).]

An independently developed ontology-based similarity measure **[CITE: ontology-based similarity measure referenced in your PDF’s Section 8 / reference [6]]** could evaluate how well semantic neighborhoods are preserved

and how severe token collisions are, and could potentially be folded into the marginal-gain computation itself, e.g. as a facility-location-style term – provided the submodularity and monotonicity properties the greedy guarantee depends on (Section 3.2) are re-verified for the modified objective.